HS PF

Programmieren in Python

Module, Funktionen, Decorators

Prof. Dr.-Ing. Stefan Kray

Module

- Python-Skripte können stets als Module importiert werden
- Beispiel modul.py

```
a=3
b=5



print("Test modul.py: a+b=",a+b)
```

Module importieren (1)

- Jedes Python-Skript kann als Modul importiert werden
- Beispiel:

```
import modul
```

→ `Test modul.py: a+b= 8`

modul.py

```
a=3
b=5


print("Test modul.py: a+b=",a+b)
```

Module importieren (2)

- Jedes Python-Skript kann als Modul importiert werden
- Beispiel:

```
import modul
```

- Zugriff auf a und b:

```
a
Traceback (most recent call last):

  Input In [8] in <cell line: 1>
    a

NameError: name 'a' is not defined
```

modul.py

```
a=3
b=5


print("Test modul.py: a+b=",a+b)
```

Module importieren (3)

- Jedes Python-Skript kann als Modul importiert werden
- Beispiel:

```
import modul
```

- Zugriff auf a und b:

```
modul.a
```

```
Out[9]: 3
```

```
modul.b
```

```
Out[10]: 5
```

modul.py

```
a=3
b=5



print("Test modul.py: a+b=",a+b)
```

Module importieren (4)

- Wie kann der Test des Moduls (Print Statement) beim Import verhindert werden?
- Spezielles Konstrukt: if __name__=="__main__"
- Beispiel modul2.py

```
a=3
b=5



if __name__=="__main__":
    print("Test modul2.py: a+b=",a+b)
```

Ausgabe: Test modul2.py: a+b= 8

Module importieren (5)

- Jedes Python-Skript kann als Modul importiert werden
- Beispiel:

```
import modul2
```

- → keine Ausgabe!
- Beim Importieren des Moduls ist das Attribut __name__ nicht auf "__main__" gesetzt, daher wird das print-Statement beim Import NICHT ausgeführt

- Anwendung: z.B. Modultests, nur wenn ein Modul direkt ausgeführt wird, laufen die Tests. Beim Import des Moduls werden die Tests nicht ausgeführt

modul2.py

```
a=3
b=5

if __name__=="__main__":
    print("Test modul2.py:a+b=",a+b)
```

Funktionen in Python

- In Python (und anderen Sprachen) lagert man Code in Funktionen aus
- Funktionen kapseln Code-Blöcke
- Funktionen können Parameter übergeben werden
- Funktionen geben Werte zurück

Funktionen (1)

- Funktionen werden durch das Schlüsselwort „def“ definiert:

```
def test():
    print("Funktionsaufruf Test")

test()
```

Funktionen (2)

- Der Vorteil von Funktionen:
  - Code-Kapselung
  - Parameter und Code auslagern
  - Parameter: in Klammern in Funktion
  - Beispiel:

```
def test(a):
    print("Wert:",a)

test(3)
test("Hallo")
test([1,2,"Test"])
```

  - Anmerkung: Man sollte sich gut überlegen, Variablen für verschiedene Datentypen zu verwenden

Funktionen (3)

- Funktionen können zwar unter bestimmten Bedingungen äußere Variablen ändern, dies sollte jedoch vermieden werden!!!!
- Funktionsparameter NUR IN DIE Funktion (anders als in C/C++)!
- Gegenbeispiel mit schlechtem Code:

```
def test(a):
    print("Wert:",a)
    a=[1,2,3]
    a.append(23)

l=[1,2,4]
test(l)
print("Wert nach Aufruf:",l)
```

- Was gibt folgender Code aus?
- Was gibt der Code aus, wenn die Zeile a=[1,2,3] entfernt wird?

Funktionen (4)

- In Python gibt man Funktionswerte per „return“ zurück
- → Funktionsparameter: Eingang in Funktion; Return: Ausgang aus Funktion
- Es dürfen auch mehrere Werte (als Tuple) zurückgegeben werden

```
def func(x, a, b, c):
    return a * np.exp(-b * x) + c

def func(x, a, b, c):
    return a * np.exp(-b * x) + c,42
```

Beispiel (1)

- modul3.py: Was gibt folgender Code aus:

```
def f1(a,b):
    return a*b
mal=f1

def f2(a,b):
    return a+b

def f3(a,b):
    return a-b

def f4(a,b):
    return mal(a,1/b)


if __name__=="__main__":
    print(f1(4,2))
    print(f2(4,2))
    print(f3(4,2))
    print(f4(4,2))
```

Beispiel (2)

- Was gibt dann folgender Code aus:

```
import modul3

def f5(a,b):
    return a**b

modul3.f2=modul3.f1
modul3.f1=modul3.f2
modul3.f4=f5
modul3.f5=modul3.f4


print(modul3.f1(4,2))
print(modul3.f2(4,2))
print(modul3.f3(4,2))
print(modul3.f4(4,2))
print(modul3.f5(4,2))
```

modul3.py

```
def f1(a,b):
    return a*b

def f2(a,b):
    return a+b

def f3(a,b):
    return a-b

def f4(a,b):
    return a*1/b


if __name__=="__main__":
    print(f1(4,2))
    print(f2(4,2))
    print(f3(4,2))
    print(f4(4,2))
```

Funktionen (5)

- Python unterstützt auch vordefinierte und optionale Parameter
- Beispiel vordefinierte Parameter:

```
def test(a,b=42):
    print("a=",a)
    print("b=",b)

test(12)
test(12,33)
test(12,b=33)
```

Funktionen (6)

- Python unterstützt auch vordefinierte und Keyword-Parameter
- Keyword-Parameter werden in Dictionary kwargs aufgenommen. Beispiel:

```
def test(a,b=42,**kwargs):
    print("a=",a)
    print("b=",b)
    if len(kwargs)>0:
        if "c" in kwargs:
            print("c=",kwargs["c"])
        print('kwargs (dictionary):',kwargs)
```

test(12) → **a=12,b=42**

test(12,33) → **a=12, b=33**

test(12,b=33) → **a=12, b=33**

test(12,33,c=43) → **a=12, b=33, c=43, kwargs (dictionary): {'c': 43}**

test(12,33,c=43,d=99) → **a=12, b=33, c=43, kwargs (dictionary): {'c': 43, 'd': 99}**

Funktionen (7)

- Fixe und Keyword-Parameter in unbekannter Anzahl kombinierbar

```
def test2(*args,**kwargs):
    print("args=",args)
    print("kwargs=",kwargs)
```

`test2(3)` → **args= (3,); kwargs= {}**

`test2(3,5)` → **args= (3,5); kwargs= {}**

`test2(3,b=5,c=42)` → **args= (3,); kwargs= {'b': 5, 'c': 42}**

Lambda-Funktionen

- Lambda-Funktionen sind anonyme Funktionen

```
miau=lambda: print("miau")
miau()
```

- Parameter sind ebenfalls in Lambda-Funktionen möglich

```
miau=lambda x: print(x)
miau("miaumiau")
```

- Lambda-Funktionen werden z.B. für Hilfsfunktionen eingesetzt oder für einfache Umwandlungen von Datentypen

Aufgabe modul4.py (Teil 1)

- Schreiben Sie eine Funktion "compute1(x)", die die beiden folgenden Werte berechnet und zurückgibt:

$$2x^2 + 5$$
$$2x - 1$$

- Schreiben Sie eine Funktion "compute2(x)", die die beiden folgenden Werte berechnet und zurückgibt:

$$3x^2 + 1$$
$$3x - 2$$

- Schreiben Sie nun eine Funktion "logging(f,x)", die ein Funktion und den Funktionswert x als Eingabe nimmt. "logging" gibt dann den Eingabewert x aus, führt die Funktion aus und gibt anschließend die beiden Ausgabewerte der Funktion f aus.

- Testen Sie die mit "logging(compute1,3)" und "logging(compute2,3)"

Aufgabe modul4test.py (Teil 2)

- Schreiben Sie nun ein Programm modul4test.py
- importieren Sie modul4.py
- Definieren Sie eine Lambda Funktion f, die compute1(x+5) berechnet
  - Was gibt `print(f(3))` aus?

- Definieren Sie eine Funktion f2, die berechnet:
  - compute1(a) falls f2(a=...) aufgerufen wird
  - compute2(b) falls f2(b=...) aufgerufen wird
  - compute1(0) falls f2 mit anderen Parametern aufgerufen wird

- Testen Sie mit

```
print(f(3))
print(f2(a=3))
print(f2(b=3))
print(f2(c=3))
```

Funktionen in Funktionen

- Funktionen können auch in Funktionen definiert werden
- Beispiel:

```
def hi():
    print("now you are inside the hi() function")
    def greet():
        return "now you are in the greet() function"
    def welcome():
        return "now you are in the welcome() function"
    print(greet())
    print(welcome())
    print("now you are back in the hi() function")
hi()
```

Was ist die Ausgabe?

Python Decorators (1)

- Decorator sind "Wrapper" für Funktionen
- Beispiel: Die Logging-Funktion aus Modul4 ist eine Art Wrapper für bestimmte andere Funktionen, die diesen eine Logging-Methodik (Ausgabe von Debug-Parametern) hinzufügt
- Beispiel:

```
def my_decorator(func):
    def wrapper():
        print("Something is happening before the function is called.")
        result = func()
        print("Something is happening after the function is called.")
        return result
    return wrapper
```

Python Decorators (2)

- Beispiel:

```
def my_decorator(func):
    def wrapper():
        print("Something is happening before the function is called.")
        result = func()
        print("Something is happening after the function is called.")
        return result
    return wrapper

f=lambda :print("Text from f")
f2=my_decorator(f)
f2()
```

Python Decorators (3)

- Beispiel etwas allgemeiner:

```
def my_decorator2(func):
    def wrapper(*args,**kwargs):
        print("Something is happening before the function is called.")
        result=func(*args,**kwargs)
        print("Something is happening after the function is called.")
     return result
    return wrapper

f=lambda x:print("Text from f,x=",x)
f2=my_decorator2(f)
f2(3)
```

Something is happening before the function is called.

Text from f,x= 3

Something is happening after the function is called.

Python Decorators (4)

- Beispiel etwas allgemeiner:

```
def my_decorator2(func):
    def wrapper(*args,**kwargs):
        print("Something is happening before the function is called.")
        result=func(*args,**kwargs)
        print("Something is happening after the function is called.")
     return result
    return wrapper
```

Unterschied zu vorher: wrapper nimmt nun beliebig viele Argumente auf und gibt sie an die Funktion weiter

Python Decorators (5)

```
def my_decorator2(func):
    def wrapper(*args,**kwargs):
        print("Something is happening before the function is called.")
        result=func(*args,**kwargs)
        print("Something is happening after the function is called.")
     return result
    return wrapper

def testfunktion(x):
    print("Wert aus Testfunktion: x=",x)

f2=my_decorator2(testfunktion)
f2(3)
```

Aufruf von testfunktion mit my_decorato2 ist umständlich!

Python Decorators (6)

```
def my_decorator2(func):
    def wrapper(*args,**kwargs):
        print("Something is happening before the function is called.")
        result=func(*args,**kwargs)
        print("Something is happening after the function is called.")
     return result
    return wrapper
```

Abkürzung:

```
@my_decorator2
def testfunktion(x):
    print("Wert aus Testfunktion: x=",x)
```

Abkürzende Schreibweise mit @funktionsname_des_decorators

```
testfunktion(3)
```
→

Something is happening before the function is called.

Wert aus Testfunktion: x= 3

Something is happening after the function is called.

Aufgabe decorator_test

- Schreiben Sie ein Programm "decorator_test.py"
- Entwerfen Sie einen Logger als Decorator-Funktion:
  - der *args und **kwargs-Parameter annimmt und per Print ausgibt
  - der die berechneten Ergebniswerte per Print ausgibt

- Testen Sie Ihren Logger mit:

```
@logger
def dectest(x,a=42):
    return x+a,x-a

dectest(3,a=43)
```

→

args: (3,)

kwargs: {'a': 43}